

# **Assignment 3**

## **Integration of Input Memory, Read Address Generator and MAC unit**

Submitted by: KUSUMURU SURESH KUMAR

([kusumurusureshkumar21@gmail.com](mailto:kusumurusureshkumar21@gmail.com))

Find codes at: [https://github.com/suresh-vlsi/IITKGP\\_assignment](https://github.com/suresh-vlsi/IITKGP_assignment)

## Part1:

### 1(i) Reset, start, weight loading

#### 1. cnn\_weight\_controller.sv

```
`timescale 1ns/1ps

module cnn_weight_controller (
    input wire      clk,
    input wire      rst_n,
    input wire      start,

    output reg      wt_wr_en,
    output reg [3:0] wt_addr,
    output reg [7:0] wt_data,
    output reg      scan_done,
    output reg [1:0] state
);

    localparam IDLE = 2'd0;
    localparam LOAD = 2'd1;
    localparam DONE = 2'd2;

    reg [3:0] count;

    // Example 3x3 kernel:
    //  1  0 -1
    //  1  0 -1
    //  1  0 -1
    function [7:0] kernel_value;
        input [3:0] index;
        begin
            case (index)
                0: kernel_value = 8'd1;
                1: kernel_value = 8'd0;
                2: kernel_value = -8'sd1;
                3: kernel_value = 8'd1;
                4: kernel_value = 8'd0;
                5: kernel_value = -8'sd1;
                6: kernel_value = 8'd1;
                7: kernel_value = 8'd0;
                8: kernel_value = -8'sd1;
                default: kernel_value = 8'd0;
            endcase
        end
    endfunction

    always @(posedge clk) begin
        if (!rst_n) begin
            state      <= IDLE;
            count      <= 0;
            wt_wr_en    <= 0;
            wt_addr     <= 0;
            wt_data     <= 0;
            scan_done   <= 0;
        end
        else begin
            // Default: write enable and done are pulses.
            wt_wr_en    <= 0;
            scan_done   <= 0;
        end
    end
endmodule
```

```

        case (state)
            IDLE: begin
                count <= 0;

                if (start)
                    state <= LOAD;
            end

            LOAD: begin
                wt_wr_en <= 1;
                wt_addr <= count;
                wt_data <= kernel_value(count);

                if (count == 8) begin
                    state <= DONE;
                end
                else begin
                    count <= count + 1'b1;
                end
            end

            DONE: begin
                scan_done <= 1;

                if (!start)
                    state <= IDLE;
            end

            default: begin
                state <= IDLE;
                count <= 0;
            end
        endcase
    end
end
endmodule

```

## 2. tb\_cnn\_weight\_controller.sv

```

`timescale 1ns/1ps
module tb_cnn_weight_controller;
    reg clk;
    reg rst_n;
    reg start;
    wire wt_wr_en;
    wire [3:0] wt_addr;
    wire [7:0] wt_data;
    wire scan_done;
    wire [1:0] state;
    integer errors;
    integer writes;
    cnn_weight_controller dut (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .wt_wr_en(wt_wr_en),
        .wt_addr(wt_addr),
        .wt_data(wt_data),
        .scan_done(scan_done),
        .state(state)
    );
endmodule

```

```

    );
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
initial begin
    $dumpfile("cnn_weight_controller.vcd");
    $dumpvars(0, tb_cnn_weight_controller);
errors = 0;
    writes = 0;
    rst_n = 0;
    start = 0;
// Hold reset for two clock edges.
    repeat (2) @(negedge clk);
    rst_n = 1;
// Start the weight-loading sequence.
    @(negedge clk);
    start = 1;
// Observe all nine weight writes.
    while (writes<9) begin
        @(negedge clk);
if (wt_wr_en) begin
            $display(
                "WEIGHT WRITE: addr=%0d data=0x%02h",
                wt_addr, wt_data
            );
if (wt_addr != writes[3:0])
                errors = errors + 1;
case (writes)
                    0, 3, 6:
                        if (wt_data != 8'h01)
                            errors = errors + 1;
                    1, 4, 7:
                        if (wt_data != 8'h00)
                            errors = errors + 1;
                    2, 5, 8:
                        if (wt_data != 8'hFF)
                            errors = errors + 1;
                endcase
writes = writes + 1;
            end
        end
// Wait for completion.
        wait (scan_done === 1'b1);
        $display("scan_done asserted.");
if (errors == 0)
            $display("PART 1(i) CONTROLLER TEST PASSED");
        else
            $display("PART 1(i) FAILED: %0d errors", errors);
start = 0;
        repeat (2) @(negedge clk);
        $finish;
    end
// Watchdog to prevent a hung simulation.
    initial begin
        #2000;
        $display("ERROR: TESTBENCH TIMEOUT");
        $finish;
    end
endmodule

```

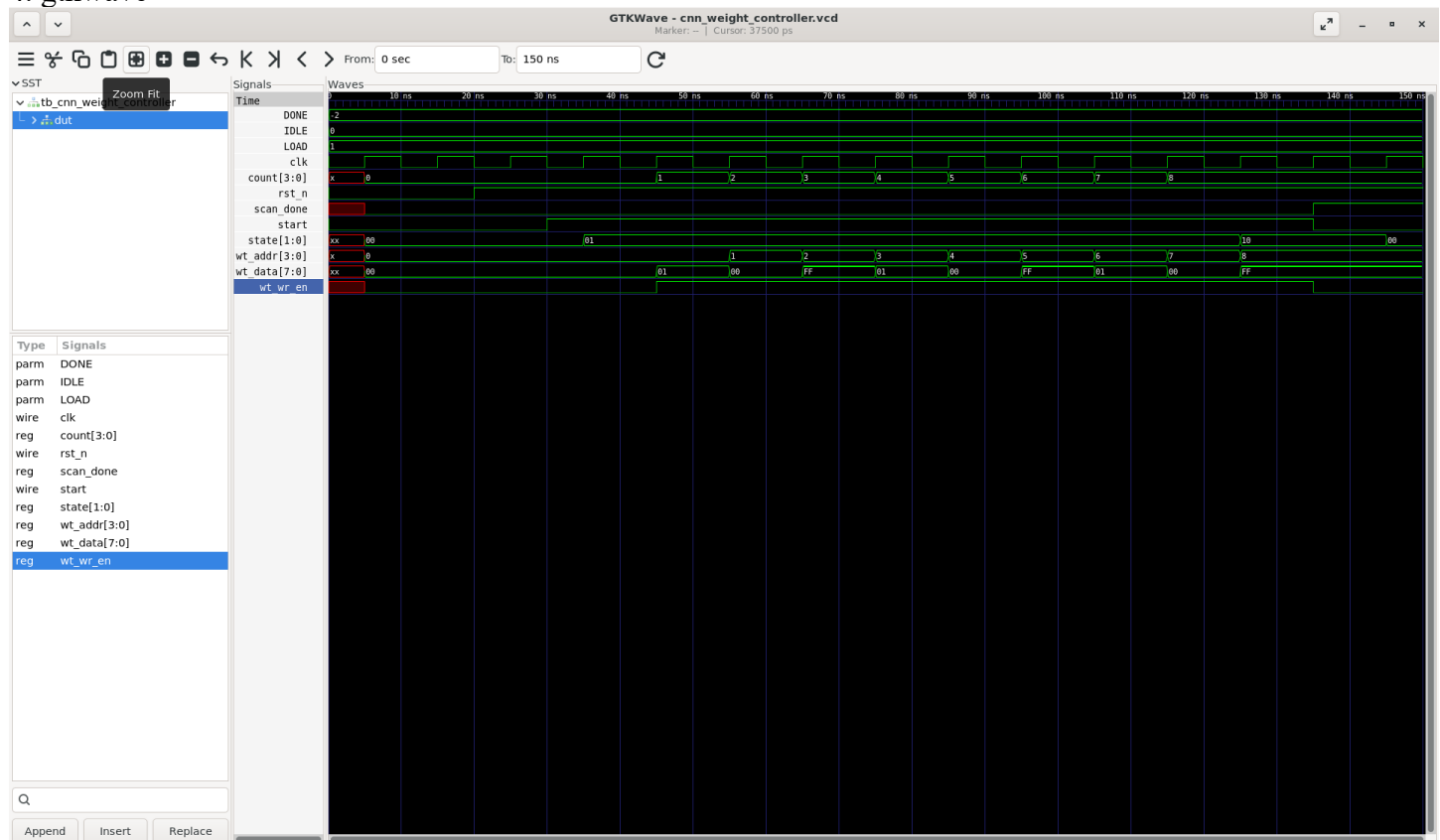
### 3. terminal

```

suresh@lsid@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/IITKGP_assignment/3$ iverilog -g2012 -Wall \
-s tb_cnn_weight_controller \
-o /tmp/part1_weight.out \
cnn_weight_controller.sv \
tb_cnn_weight_controller.sv \
&& vvp /tmp/part1_weight.out
VCD info: dumpfile cnn_weight_controller.vcd opened for output.
WEIGHT WRITE: addr=0 data=0x01
WEIGHT WRITE: addr=1 data=0x00
WEIGHT WRITE: addr=2 data=0xff
WEIGHT WRITE: addr=3 data=0x01
WEIGHT WRITE: addr=4 data=0x00
WEIGHT WRITE: addr=5 data=0xff
WEIGHT WRITE: addr=6 data=0x01
WEIGHT WRITE: addr=7 data=0x00
WEIGHT WRITE: addr=8 data=0xff
scan_done asserted.
PART 1(i) CONTROLLER TEST PASSED

```

#### 4. gtkwave



## 1(ii) Verify preloaded image pixels in SRAM

### 1. cnn\_input\_sram.sv

```
`timescale 1ns/1ps

module cnn_input_sram (
    input wire      clk,

    input wire      write_en,
    input wire [1:0] write_bank,
    input wire [13:0] write_addr,
    input wire [7:0] write_data,

    input wire      read_en,
    input wire [7:0] start_row,
    input wire [7:0] start_col,

    output reg [23:0] cnn_data,
    output reg      out_valid,

    output wire [1:0] bank0,
    output wire [1:0] bank1,
    output wire [1:0] bank2,

    output wire [13:0] addr0,
    output wire [13:0] addr1,
    output wire [13:0] addr2,

    output wire [7:0] pixel0,
    output wire [7:0] pixel1,
    output wire [7:0] pixel2
);

// =====
// 64 KB SRAM
// 4 banks x 16 KB
// =====

reg [7:0] mem [0:65535];

// =====
// WRITE
//
// Physical address:
// bank 0 -> 0x0000 - 0x3FFF
// bank 1 -> 0x4000 - 0x7FFF
// bank 2 -> 0x8000 - 0xBFFF
// bank 3 -> 0xC000 - 0xFFFF
// =====

wire [15:0] write_phys_addr;

assign write_phys_addr =
    {write_bank, write_addr};

always @(posedge clk) begin
    if (write_en)
        mem[write_phys_addr] <= write_data;
end
```

```

// =====
// THREE-PIXEL CNN WINDOW
//
// Pixel 0 = (start_row, start_col)
// Pixel 1 = (start_row, start_col+1)
// Pixel 2 = (start_row, start_col+2)
// =====

wire [8:0] col0;
wire [8:0] col1;
wire [8:0] col2;

assign col0 = {1'b0,start_col};
assign col1 = {1'b0,start_col} + 9'd1;
assign col2 = {1'b0,start_col} + 9'd2;

// =====
// STAGGERED BANK SELECTION
//
// column 0 -> bank 0
// column 1 -> bank 1
// column 2 -> bank 2
// column 3 -> bank 3
// column 4 -> bank 0
// ...
// =====

assign bank0 = col0[1:0];
assign bank1 = col1[1:0];
assign bank2 = col2[1:0];

// =====
// LOCAL ADDRESS INSIDE EACH BANK
//
// 256 rows x 64 bytes
//
// address = row*64 + column/4
// =====

assign addr0 =
    (start_row << 6) + col0[8:2];

assign addr1 =
    (start_row << 6) + col1[8:2];

assign addr2 =
    (start_row << 6) + col2[8:2];

// =====
// PHYSICAL ADDRESSES
// =====

wire [15:0] phys0;
wire [15:0] phys1;
wire [15:0] phys2;

assign phys0 = {bank0,addr0};
assign phys1 = {bank1,addr1};
assign phys2 = {bank2,addr2};

// =====

```

```

// READ PIXELS
// =====

assign pixel0 = mem[phys0];
assign pixel1 = mem[phys1];
assign pixel2 = mem[phys2];

// =====
// 24-BIT OUTPUT
// =====

always @(posedge clk) begin

    if (read_en) begin
        cnn_data <= {pixel0,pixel1,pixel2};
        out_valid <= 1'b1;
    end
    else begin
        cnn_data <= 24'h000000;
        out_valid <= 1'b0;
    end

end

endmodule

```

## 2. tb\_cnn\_input\_sram\_data\_check.sv

```

`timescale 1ns/1ps

module tb_cnn_input_sram_data_check;

    reg clk;
    reg write_en;
    reg [1:0] write_bank;
    reg [13:0] write_addr;
    reg [7:0] write_data;
    reg read_en;
    reg [7:0] start_row;
    reg [7:0] start_col;

    wire [23:0] cnn_data;
    wire out_valid;

    wire [1:0] bank0, bank1, bank2;
    wire [13:0] addr0, addr1, addr2;
    wire [7:0] pixel0, pixel1, pixel2;

    integer errors;

    cnn_input_sram dut (
        .clk(clk),
        .write_en(write_en),
        .write_bank(write_bank),
        .write_addr(write_addr),
        .write_data(write_data),
        .read_en(read_en),
        .start_row(start_row),
        .start_col(start_col),
        .cnn_data(cnn_data),
        .out_valid(out_valid),
        .bank0(bank0),

```

```

        .bank1(bank1),
        .bank2(bank2),
        .addr0(addr0),
        .addr1(addr1),
        .addr2(addr2),
        .pixel0(pixel0),
        .pixel1(pixel1),
        .pixel2(pixel2)
    );

    always #5 clk = ~clk;

    task write_pixel;
        input [1:0] bank;
        input [13:0] addr;
        input [7:0] data;
        begin
            @(negedge clk);
            write_en = 1'b1;
            write_bank = bank;
            write_addr = addr;
            write_data = data;

            @(negedge clk);
            write_en = 1'b0;
        end
    endtask

    initial begin
        clk = 0;
        write_en = 0;
        write_bank = 0;
        write_addr = 0;
        write_data = 0;
        read_en = 0;
        start_row = 0;
        start_col = 0;
        errors = 0;

        $dumpfile("input_data_check.vcd");
        $dumpvars(0, tb_cnn_input_sram_data_check);

        $display("=== INPUT SRAM PIXEL DATA CHECK ===");

        // Initialize three consecutive pixels in banks 0, 1, 2.
        write_pixel(0, 0, 8'h11);
        write_pixel(1, 0, 8'h22);
        write_pixel(2, 0, 8'h33);

        start_row = 0;
        start_col = 0;

        @(negedge clk);
        read_en = 1'b1;

        @(posedge clk);
        #1;

        if (out_valid !== 1'b1 ||
            cnn_data !== 24'h112233) begin
            $display(

```

```

        "FAIL: data=%h valid=%b expected=112233 valid=1",
        cnn_data, out_valid
    );
    errors = errors + 1;
end else begin
    $display(
        "PASS: data=%h valid=%b",
        cnn_data, out_valid
    );
end

@(negedge clk);
read_en = 1'b0;

@(posedge clk);
#1;

if (out_valid !== 1'b0 ||
    cnn_data !== 24'h000000) begin
    $display("FAIL: read-disabled behavior");
    errors = errors + 1;
end else begin
    $display("PASS: read-disabled behavior");
end

if (errors == 0)
    $display("INPUT DATA CHECK PASSED: 2/2 checks");
else
    $display("INPUT DATA CHECK FAILED: %0d errors", errors);

$finish;
end
endmodule

```

### 3. tb\_cnn\_input\_sram\_full\_image.sv

```

`timescale 1ns/1ps

module tb_cnn_input_sram_full_image;

    reg clk = 0;
    reg write_en = 0;
    reg [1:0] write_bank = 0;
    reg [13:0] write_addr = 0;
    reg [7:0] write_data = 0;
    reg read_en = 0;
    reg [7:0] start_row = 0;
    reg [7:0] start_col = 0;

    wire [23:0] cnn_data;
    wire out_valid;
    wire [1:0] bank0, bank1, bank2;
    wire [13:0] addr0, addr1, addr2;
    wire [7:0] pixel0, pixel1, pixel2;

    integer r, c, linear;
    integer errors = 0;
    integer writes = 0;
    integer windows = 0;
    integer checks = 0;

```

```

cnn_input_sram dut (
    .clk(clk),
    .write_en(write_en),
    .write_bank(write_bank),
    .write_addr(write_addr),
    .write_data(write_data),
    .read_en(read_en),
    .start_row(start_row),
    .start_col(start_col),
    .cnn_data(cnn_data),
    .out_valid(out_valid),
    .bank0(bank0), .bank1(bank1), .bank2(bank2),
    .addr0(addr0), .addr1(addr1), .addr2(addr2),
    .pixel0(pixel0), .pixel1(pixel1), .pixel2(pixel2)
);

always #5 clk = ~clk;

function [7:0] pixel_value;
    input integer row;
    input integer col;
    begin
        pixel_value = (row * 17 + col) & 255;
    end
endfunction

initial begin
    $display("=== FULL IMAGE SRAM VERIFICATION ===");

    $dumpfile("cnn_input_sram_full_image.vcd");
    $dumpvars(0, tb_cnn_input_sram_full_image);

    // Write all 256 x 256 pixels.
    for (r = 0; r < 256; r = r + 1) begin
        for (c = 0; c < 256; c = c + 1) begin
            linear = r * 256 + c;

            @(negedge clk);
            write_en = 1;
            write_bank = linear % 4;
            write_addr = linear / 4;
            write_data = pixel_value(r, c);

            @(negedge clk);
            write_en = 0;
            writes = writes + 1;
        end
    end

    $display("Pixels written: %0d", writes);

    // Read every valid three-pixel window.
    // Columns 0..253 are valid on every row.
    // Columns 254..255 are valid except on the last row.
    for (r = 0; r < 256; r = r + 1) begin
        for (c = 0; c < 256; c = c + 1) begin
            if ((r < 255) || (c <= 253)) begin
                @(negedge clk);
                start_row = r;
                start_col = c;
                read_en = 1;
            end
        end
    end
end

```

```

        @(posedge clk);
        #1;

        windows = windows + 1;

        if (out_valid != 1'b1) begin
            errors = errors + 1;
            if (errors <= 10)
                $display("FAIL valid row=%0d col=%0d",
                    r, c);
        end
        else begin
            checks = checks + 1;

            if (cnn_data != {
                pixel_value(r, c),
                pixel_value(r + ((c+1)/256),
                    (c+1)%256),
                pixel_value(r + ((c+2)/256),
                    (c+2)%256)
            }) begin
                errors = errors + 1;
                if (errors <= 10)
                    $display(
                        "FAIL row=%0d col=%0d expected=%06h actual=%06h",
                        r, c,
                        {
                            pixel_value(r, c),
                            pixel_value(r + ((c+1)/256),
                                (c+1)%256),
                            pixel_value(r + ((c+2)/256),
                                (c+2)%256)
                        },
                        cnn_data
                    );
            end
        end

        @(negedge clk);
        read_en = 0;
    end
end

$display("-----");
$display("Pixels written : %0d", writes);
$display("Windows tested : %0d", windows);
$display("Data checks      : %0d", checks);
$display("Errors           : %0d", errors);

if (errors == 0)
    $display("FULL IMAGE TEST PASSED");
else
    $display("FULL IMAGE TEST FAILED");

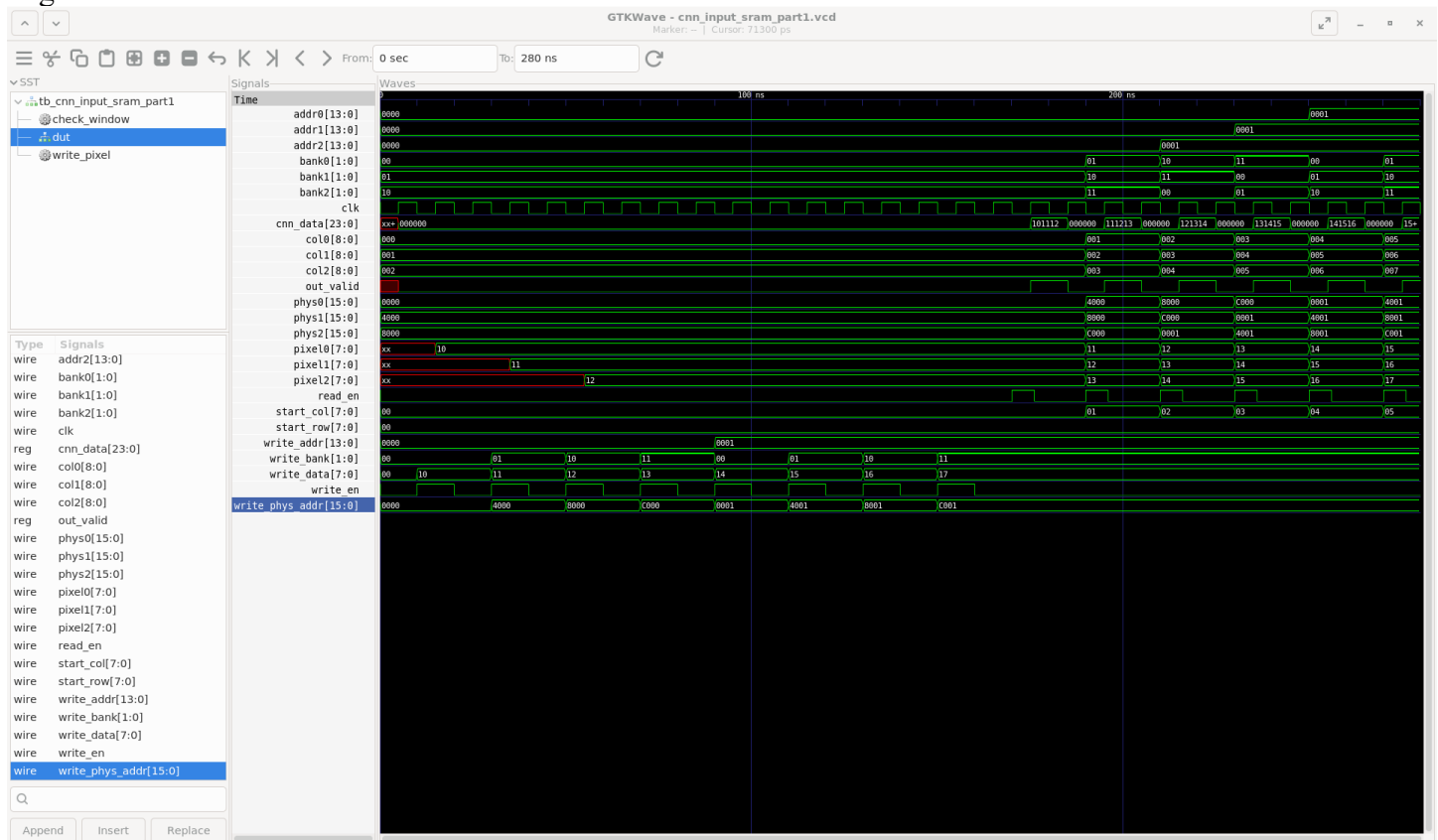
$finish;
end
endmodule

```

### 3. terminal

```
sureshvisi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/IITKGP_assignment/3$ iverilog -g2012 -Wall \  
-s tb_cnn_input_sram_data_check \  
-o /tmp/part1_data.out \  
cnn_input_sram.sv \  
tb_cnn_input_sram_data_check.sv \  
&& vvp /tmp/part1_data.out  
VCD info: dumpfile input_data_check.vcd opened for output.  
=== INPUT SRAM PIXEL DATA CHECK ===  
PASS: data=112233 valid=1  
PASS: read-disabled behavior  
INPUT DATA CHECK PASSED: 2/2 checks  
tb_cnn_input_sram_data_check.sv:127: $finish called at 86000 (1ps)  
sureshvisi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/IITKGP_assignment/3$ iverilog -g2012 -Wall \  
-s tb_cnn_input_sram_full_image \  
-o /tmp/part1_full_image.out \  
cnn_input_sram.sv \  
tb_cnn_input_sram_full_image.sv \  
&& vvp /tmp/part1_full_image.out  
=== FULL IMAGE SRAM VERIFICATION ===  
VCD info: dumpfile cnn_input_sram_full_image.vcd opened for output.  
Pixels written: 65536  
-----  
Pixels written : 65536  
Windows tested : 65534  
Data checks : 65534  
Errors : 0  
FULL IMAGE TEST PASSED  
tb_cnn_input_sram_full_image.sv:143: $finish called at 2621400000 (1ps)  
sureshvisi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/IITKGP_assignment/3$
```

### 4. gtkwave



### 1(iii) Verify row/column address sequence

#### 1. cnn\_address\_generator.sv

```
`timescale 1ns/1ps

module cnn_address_generator #(
    parameter [7:0] LAST_ROW = 8'd255,
    parameter [7:0] LAST_COL = 8'd253
)(
    input wire      clk,
    input wire      rst_n,
    input wire      start,

    output wire      read_en,
    output reg [7:0] row,
    output reg [7:0] col,
    output reg       scan_done,

    output wire [1:0] bank0,
    output wire [1:0] bank1,
    output wire [1:0] bank2,

    output wire [13:0] addr0,
    output wire [13:0] addr1,
    output wire [13:0] addr2
);

    reg active;

    wire [7:0] col1;
    wire [7:0] col2;

    // A 3-pixel horizontal window must remain within one row.
    assign col1 = col + 8'd1;
    assign col2 = col + 8'd2;

    assign read_en = active;

    // Cyclic interleaving: column modulo 4 selects the bank.
    assign bank0 = col[1:0];
    assign bank1 = col1[1:0];
    assign bank2 = col2[1:0];

    // Local byte address = row*64 + column/4.
    assign addr0 = {row, 6'b0} + {6'b0, col[7:2]};
    assign addr1 = {row, 6'b0} + {6'b0, col1[7:2]};
    assign addr2 = {row, 6'b0} + {6'b0, col2[7:2]};

    always @(posedge clk) begin
        if (!rst_n) begin
            active    <= 1'b0;
            row       <= 8'd0;
            col       <= 8'd0;
            scan_done <= 1'b0;
        end
        else begin
            scan_done <= 1'b0;

            if (!active) begin
                if (start) begin
                    row    <= 8'd0;

```

```

                col    <= 8'd0;
                active <= 1'b1;
            end
        end
    else begin
        if (col < LAST_COL) begin
            col <= col + 8'd1;
        end
        else if (row < LAST_ROW) begin
            col <= 8'd0;
            row <= row + 8'd1;
        end
        else begin
            active    <= 1'b0;
            scan_done <= 1'b1;
        end
    end
end
end
end
endmodule

```

## 2. tb\_cnn\_address\_generator.sv

```

`timescale 1ns/1ps
module tb_cnn_address_generator;
    reg clk;
    reg rst_n;
    reg start;
    wire read_en;
    wire [7:0] row, col;
    wire scan_done;
    wire [1:0] bank0, bank1, bank2;
    wire [13:0] addr0, addr1, addr2;
    integer errors;
    integer windows;
    reg finished;
    cnn_address_generator #(
        .LAST_ROW(8'd0),
        .LAST_COL(8'd7)
    ) dut (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .read_en(read_en),
        .row(row),
        .col(col),
        .scan_done(scan_done),
        .bank0(bank0),
        .bank1(bank1),
        .bank2(bank2),
        .addr0(addr0),
        .addr1(addr1),
        .addr2(addr2)
    );
    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end
    initial begin
        errors = 0;
        windows = 0;
    end
endmodule

```

```

        finished = 0;
rst_n = 0;
    start = 0;
$dumpfile("cnn_address_generator.vcd");
    $dumpvars(0, tb_cnn_address_generator);
repeat (2) @(negedge clk);
    rst_n = 1;
@(negedge clk);
    start = 1;
@(negedge clk);
    start = 0;
// Sample on falling edges, away from counter updates.
    while (!scan_done && windows<20) begin
        if (read_en) begin
            $display(
                "Window %0d: row=%0d col=%0d banks=%0d,%0d,%0d addr=%0d,%0d,%0d",
                windows, row, col,
                bank0, bank1, bank2,
                addr0, addr1, addr2
            );
            if (row !== 8'd0 || col !== windows)
                errors = errors + 1;
            if (bank0 !== (windows % 4) ||
                bank1 !== ((windows + 1) % 4) ||
                bank2 !== ((windows + 2) % 4))
                errors = errors + 1;
            if (addr0 !== (windows / 4) ||
                addr1 !== ((windows + 1) / 4) ||
                addr2 !== ((windows + 2) / 4))
                errors = errors + 1;
            windows = windows + 1;
        end
    @(negedge clk);
    end
    if (!scan_done) begin
        $display("FAIL: scan did not complete");
        errors = errors + 1;
    end
    if (windows != 8) begin
        $display("FAIL: expected 8 windows, got %0d", windows);
        errors = errors + 1;
    end
    if (errors == 0)
        $display("ADDRESS GENERATOR TEST PASSED: %0d windows", windows);
    else
        $display("ADDRESS GENERATOR TEST FAILED: %0d errors", errors);
    finished = 1;
    $finish;
end
// Watchdog
initial begin
    #5000;
    if (!finished) begin
        $display("ERROR: TESTBENCH TIMEOUT");
        $finish;
    end
end
endmodule

```

### 3.tb cnn address generator scan fixed.sv

```
`timescale 1ns/1ps
```

```

module tb_cnn_address_generator_scan_fixed;

    reg clk = 0;
    reg rst_n = 0;
    reg start = 0;

    wire read_en;
    wire [7:0] row, col;
    wire scan_done;
    wire [1:0] bank0, bank1, bank2;
    wire [13:0] addr0, addr1, addr2;

    integer windows = 0;
    integer errors = 0;
    integer timeout = 0;
    integer row_windows [0:255];
    integer i;

    cnn_address_generator dut (
        .clk(clk), .rst_n(rst_n), .start(start),
        .read_en(read_en), .row(row), .col(col),
        .scan_done(scan_done),
        .bank0(bank0), .bank1(bank1), .bank2(bank2),
        .addr0(addr0), .addr1(addr1), .addr2(addr2)
    );

    always #5 clk = ~clk;

    initial begin
        for (i = 0; i < 256; i = i + 1)
            row_windows[i] = 0;

        repeat (2) @(negedge clk);
        rst_n = 1;
        @(negedge clk);
        start = 1;
        @(negedge clk);
        start = 0;

        // Sample before the next active clock edge updates row/col.
        while (!scan_done && timeout < 70000) begin
            @(negedge clk);
            timeout = timeout + 1;

            if (read_en) begin
                windows = windows + 1;
                row_windows[row] = row_windows[row] + 1;

                if (col > 253) begin
                    $display("FAIL invalid column: row=%0d col=%0d",
                        row, col);
                    errors = errors + 1;
                end
            end
        end

        for (i = 0; i < 256; i = i + 1) begin
            if (row_windows[i] != 254) begin
                $display("ROW ERROR row=%0d count=%0d",
                    i, row_windows[i]);
            end
        end
    end
endmodule

```

```

        errors = errors + 1;
    end
end

$display("Windows observed: %0d (expected 65024)", windows);

if (windows != 65024) begin
    $display("FAIL: window count mismatch");
    errors = errors + 1;
end

if (!scan_done) begin
    $display("FAIL: scan did not complete");
    errors = errors + 1;
end

if (errors == 0)
    $display("FULL SCAN TEST PASSED");
else
    $display("FULL SCAN TEST FAILED: %0d errors", errors);

$finish;
end

endmodule

```

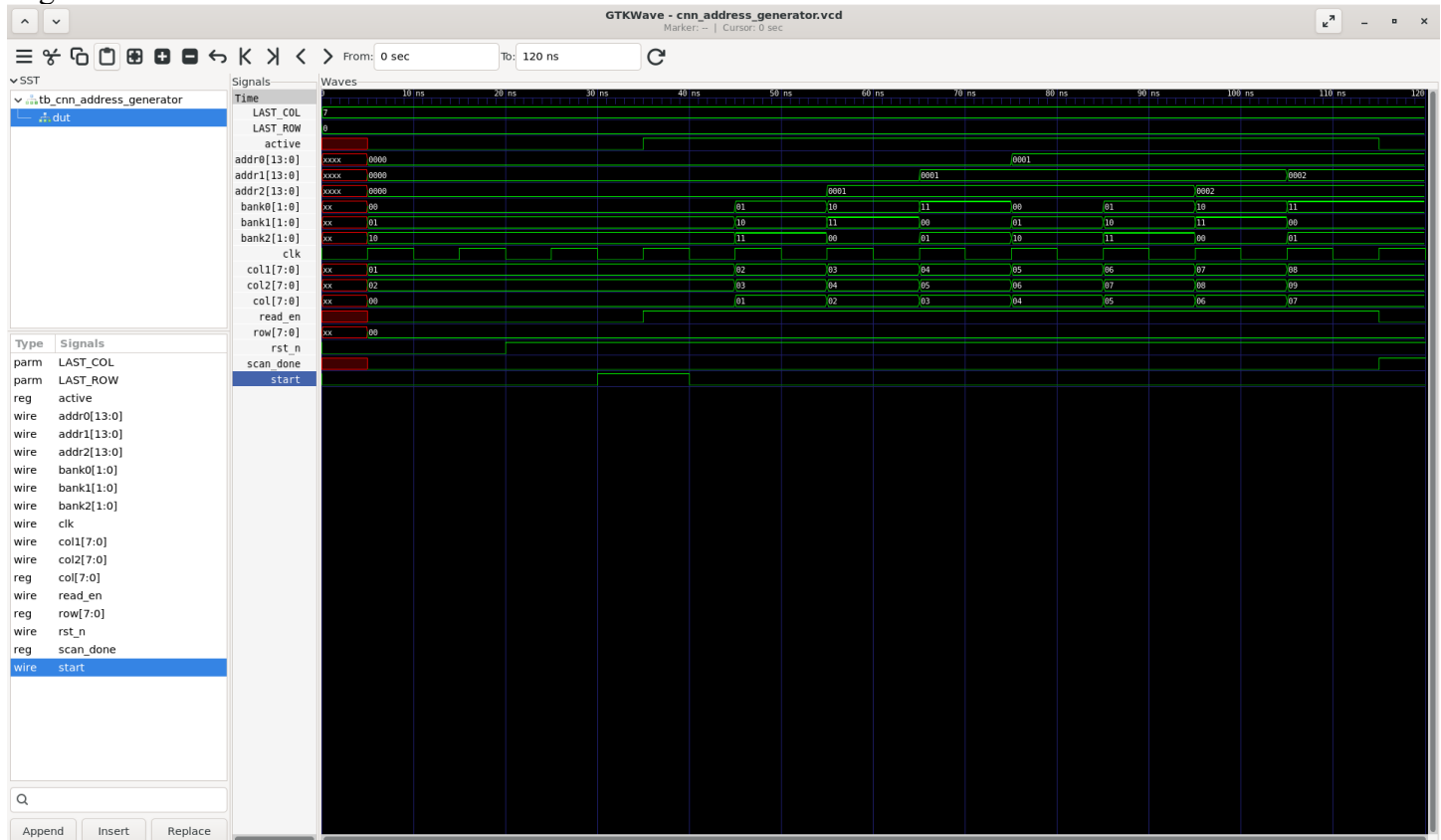
#### 4. terminal

```

sureshvlasi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/IITKGP_assignment/3$ iverilog -g2012 -Wall \
-s tb_cnn_address_generator \
-o /tmp/part1_address.out \
cnn_address_generator.sv \
tb_cnn_address_generator.sv \
&& vvp /tmp/part1_address.out
VCD info: dumpfile cnn_address_generator.vcd opened for output.
Window 0: row=0 col=0 banks=0,1,2 addr=0,0,0
Window 1: row=0 col=1 banks=1,2,3 addr=0,0,0
Window 2: row=0 col=2 banks=2,3,0 addr=0,0,1
Window 3: row=0 col=3 banks=3,0,1 addr=0,1,1
Window 4: row=0 col=4 banks=0,1,2 addr=1,1,1
Window 5: row=0 col=5 banks=1,2,3 addr=1,1,1
Window 6: row=0 col=6 banks=2,3,0 addr=1,1,2
Window 7: row=0 col=7 banks=3,0,1 addr=1,2,2
ADDRESS GENERATOR TEST PASSED: 8 windows
tb_cnn_address_generator.sv:86: $finish called at 120000 (1ps)
sureshvlasi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/IITKGP_assignment/3$ iverilog -g2012 -Wall \
-s tb_cnn_address_generator_scan_fixed \
-o /tmp/part1_scan.out \
cnn_address_generator.sv \
tb_cnn_address_generator_scan_fixed.sv \
&& vvp /tmp/part1_scan.out
ROW ERROR row=0 count=253
Windows observed: 65023 (expected 65024)
FAIL: window count mismatch
FULL SCAN TEST FAILED: 2 errors
sureshvlasi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/IITKGP_assignment/3$

```

## 5. gtkwave



1(iv) Compare weight loading and image preload tasks

1. cnn\_weight\_controller.sv
  2. cnn\_input\_sram.sv
  2. tb\_cnn\_weight\_controller.sv
  3. tb\_cnn\_input\_sram\_data\_check.sv
  4. tb\_cnn\_input\_sram\_full\_image.sv
- (tested already above)

1(v) Verify output memory read address/data

1. cnn\_output\_sram.sv

```
`timescale 1ns/1ps

module cnn_output_sram (
    input wire      clk,

    // Write interface
    input wire      wr_en,
    input wire [7:0] wr_addr,
    input wire [23:0] wr_data,

    // Read interface
    input wire      rd_en,
    input wire [7:0] rd_addr,
    output reg [23:0] rd_data,
    output reg      rd_valid
);

// 256 words x 24 bits
reg [23:0] mem [0:255];
```

```

// Synchronous write and read
always @(posedge clk) begin
    if (wr_en)
        mem[wr_addr] <= wr_data;

    if (rd_en) begin
        rd_data <= mem[rd_addr];
        rd_valid <= 1'b1;
    end
    else begin
        rd_data <= 24'h000000;
        rd_valid <= 1'b0;
    end
end

endmodule

```

## 2. tb\_part1v\_output\_read.sv

```

`timescale 1ns/1ps

module tb_part1v_output_read;

    reg clk = 1'b0;

    reg      wr_en    = 1'b0;
    reg [7:0] wr_addr = 8'd0;
    reg [23:0] wr_data = 24'd0;

    reg      rd_en    = 1'b0;
    reg [7:0] rd_addr = 8'd0;

    wire [23:0] rd_data;
    wire      rd_valid;

    integer errors = 0;
    integer checks = 0;

    // Expected MAC results for four known pixel windows.
    reg [23:0] expected [0:3];

    cnn_output_sram dut (
        .clk      (clk),
        .wr_en    (wr_en),
        .wr_addr  (wr_addr),
        .wr_data  (wr_data),
        .rd_en    (rd_en),
        .rd_addr  (rd_addr),
        .rd_data  (rd_data),
        .rd_valid (rd_valid)
    );

    always #5 clk = ~clk;

    // Write one 24-bit expected convolution result.
    task write_result;
        input [7:0] address;
        input [23:0] value;
        begin
            @(negedge clk);

```

```

        wr_en   = 1'b1;
        wr_addr = address;
        wr_data = value;

        @(posedge clk);
        #1;

        $display(
            "WRITE addr=%0d data=%0d (0x%06h)",
            address, value, value
        );

        @(negedge clk);
        wr_en = 1'b0;
    end
endtask

// Read one location and compare address-associated data.
task check_result;
    input [7:0]  address;
    input [23:0] value;
    begin
        @(negedge clk);
        rd_en   = 1'b1;
        rd_addr = address;

        // The SRAM read is synchronous.
        @(posedge clk);
        #1;

        checks = checks + 1;

        if (rd_valid !== 1'b1 ||
            rd_data  !== value) begin

            $display(
                "FAIL addr=%0d expected=%0d (0x%06h) actual=%0d (0x%06h) valid=%b",
                address, value, value,
                rd_data, rd_data, rd_valid
            );

            errors = errors + 1;
        end
        else begin
            $display(
                "PASS addr=%0d data=%0d (0x%06h) valid=%b",
                address, rd_data, rd_data, rd_valid
            );
        end

        @(negedge clk);
        rd_en = 1'b0;
    end
endtask

initial begin
    $dumpfile("part1v_output_read.vcd");
    $dumpvars(0, tb_part1v_output_read);

    // Manually calculated reference values:
    // Window 0: sum(pixel[i] * kernel[i]) = 285

```

```

// Window 1: sum(pixel[i] * kernel[i]) = 330
// Window 2: sum(pixel[i] * kernel[i]) = 375
// Window 3: sum(pixel[i] * kernel[i]) = 420

expected[0] = 24'd285;
expected[1] = 24'd330;
expected[2] = 24'd375;
expected[3] = 24'd420;

$display("=====");
$display("PART 1(v): OUTPUT SRAM READ VERIFICATION");
$display("=====");

// Store the reference convolution results.
write_result(8'd0, expected[0]);
write_result(8'd1, expected[1]);
write_result(8'd2, expected[2]);
write_result(8'd3, expected[3]);

// Read back each address and verify its output.
check_result(8'd0, expected[0]);
check_result(8'd1, expected[1]);
check_result(8'd2, expected[2]);
check_result(8'd3, expected[3]);

$display("-----");
$display("Read checks : %0d", checks);
$display("Errors      : %0d", errors);

if (errors == 0 && checks == 4)
    $display("PART 1(v) OUTPUT SRAM READ TEST PASSED");
else
    $display("PART 1(v) OUTPUT SRAM READ TEST FAILED");

$display("-----");
$finish;
end

endmodule

```

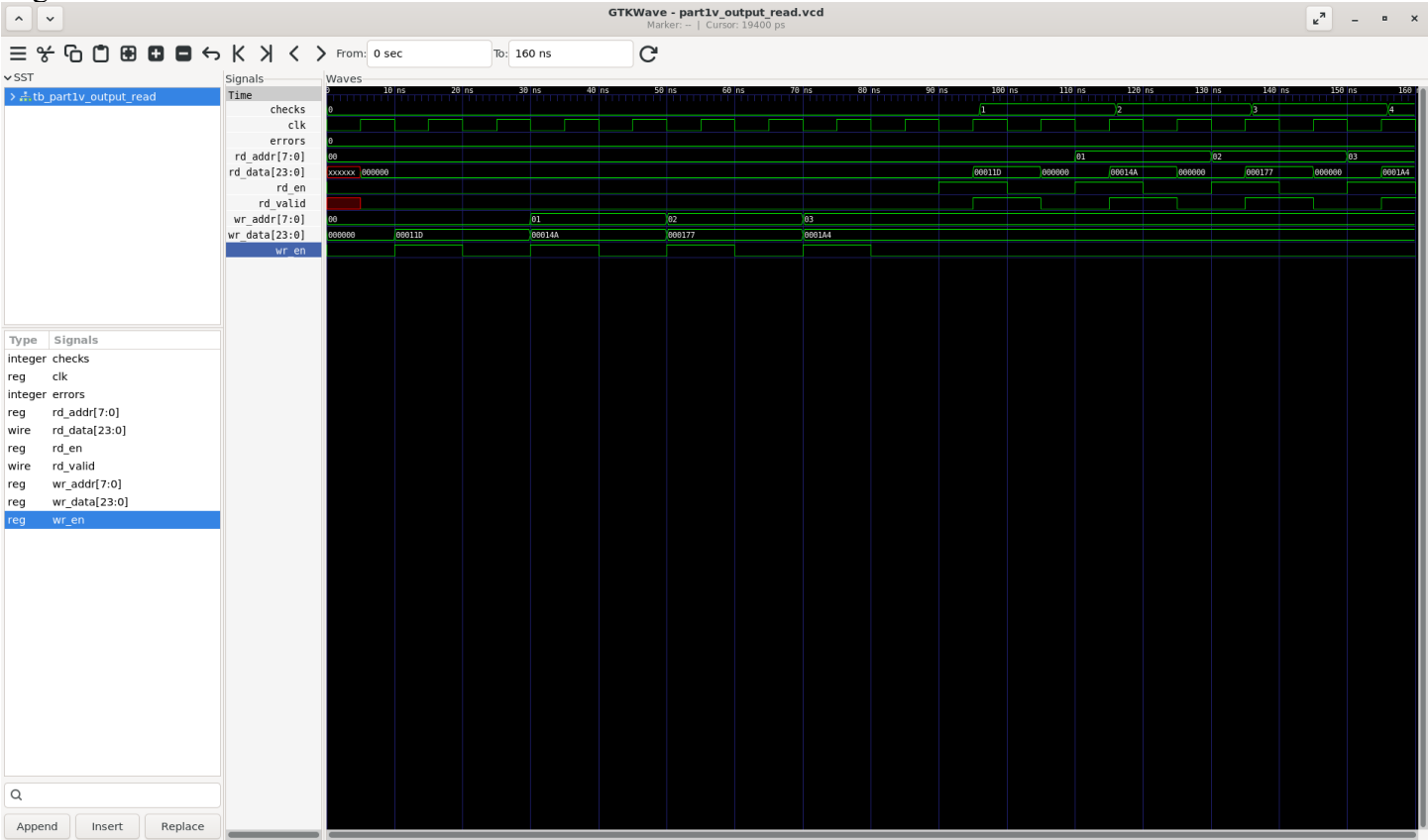
### 3. terminal

```

sureshvlisi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/IITKGP_assignment/3$ iverilog -g2012 -Wall \
-s tb_part1v_output_read \
-o /tmp/part1v.out \
cnn_output_sram.sv \
tb_part1v_output_read.sv \
&& vvp /tmp/part1v.out
VCD info: dumpfile part1v_output_read.vcd opened for output.
=====
PART 1(v): OUTPUT SRAM READ VERIFICATION
=====
WRITE addr=0 data=285 (0x00011d)
WRITE addr=1 data=330 (0x00014a)
WRITE addr=2 data=375 (0x000177)
WRITE addr=3 data=420 (0x0001a4)
PASS addr=0 data=285 (0x00011d) valid=1
PASS addr=1 data=330 (0x00014a) valid=1
PASS addr=2 data=375 (0x000177) valid=1
PASS addr=3 data=420 (0x0001a4) valid=1
-----
Read checks : 4
Errors      : 0
PART 1(v) OUTPUT SRAM READ TEST PASSED
-----
tb_part1v_output_read.sv:139: $finish called at 160000 (1ps)
sureshvlisi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/IITKGP_assignment/3$

```

4. gtkwave



## Part2:

1. cnn\_address\_generator.sv
2. cnn\_input\_sram.sv
3. tb\_part2\_address\_generator.sv

```
`timescale 1ns/1ps

module tb_part2_address_generator;

    reg clk = 0;
    reg rst_n = 0;
    reg start = 0;

    wire read_en;
    wire [7:0] row, col;
    wire scan_done;

    wire [1:0] bank0, bank1, bank2;
    wire [13:0] addr0, addr1, addr2;

    reg write_en = 0;
    reg [1:0] write_bank = 0;
    reg [13:0] write_addr = 0;
    reg [7:0] write_data = 0;

    wire [23:0] cnn_data;
    wire out_valid;

    wire [1:0] sram_bank0, sram_bank1, sram_bank2;
    wire [13:0] sram_addr0, sram_addr1, sram_addr2;
    wire [7:0] pixel0, pixel1, pixel2;

    integer errors = 0;
    integer address_checks = 0;
    integer pixel_checks = 0;
    integer cycles = 0;

    reg [7:0] sampled_row, sampled_col;
    reg [1:0] expected_bank0, expected_bank1, expected_bank2;
    reg [13:0] expected_addr0, expected_addr1, expected_addr2;

    // Deterministic reference pixel function for this test.
    function [7:0] exp_pixel;
        input [7:0] r;
        input [7:0] c;
        begin
            exp_pixel = (r * 3 + c * 5 + 17) & 8'hFF;
        end
    endfunction

    cnn_address_generator dut (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .read_en(read_en),
        .row(row),
        .col(col),
        .scan_done(scan_done),
        .bank0(bank0),
        .bank1(bank1),
        .bank2(bank2),
```

```

        .addr0(addr0),
        .addr1(addr1),
        .addr2(addr2)
    );

    cnn_input_sram sram (
        .clk(clk),
        .write_en(write_en),
        .write_bank(write_bank),
        .write_addr(write_addr),
        .write_data(write_data),
        .read_en(read_en),
        .start_row(row),
        .start_col(col),
        .cnn_data(cnn_data),
        .out_valid(out_valid),
        .bank0(sram_bank0),
        .bank1(sram_bank1),
        .bank2(sram_bank2),
        .addr0(sram_addr0),
        .addr1(sram_addr1),
        .addr2(sram_addr2),
        .pixel0(pixel0),
        .pixel1(pixel1),
        .pixel2(pixel2)
    );

    always #5 clk = ~clk;

    // Store one pixel using bank = column modulo 4.
    task load_pixel;
        input [7:0] r;
        input [7:0] c;
        reg [1:0] b;
        reg [13:0] a;
        begin
            b = c[1:0];
            a = (r << 6) + (c >> 2);

            @(negedge clk);
            write_en = 1;
            write_bank = b;
            write_addr = a;
            write_data = exp_pixel(r,c);

            @(negedge clk);
            write_en = 0;
        end
    endtask

    // Check the three bank selectors and local addresses.
    task check_address;
        input [7:0] r;
        input [7:0] c;
        reg [8:0] c1, c2;
        reg [1:0] eb0, eb1, eb2;
        reg [13:0] ea0, ea1, ea2;
        begin
            c1 = {1'b0,c} + 9'd1;
            c2 = {1'b0,c} + 9'd2;

```

```

eb0 = c[1:0];
eb1 = c1[1:0];
eb2 = c2[1:0];

ea0 = (r * 64) + (c >> 2);
ea1 = (r * 64) + (c1 >> 2);
ea2 = (r * 64) + (c2 >> 2);

address_checks = address_checks + 1;

if (bank0 != eb0 || bank1 != eb1 ||
    bank2 != eb2 ||
    addr0 != ea0 || addr1 != ea1 ||
    addr2 != ea2) begin
    $display(
        "FAIL ADDRESS row=%0d col=%0d", r, c
    );
    $display(
        "    banks actual=%0d,%0d,%0d expected=%0d,%0d,%0d",
        bank0,bank1,bank2,eb0,eb1,eb2
    );
    $display(
        "    addrs actual=%0d,%0d,%0d expected=%0d,%0d,%0d",
        addr0,addr1,addr2,ea0,ea1,ea2
    );
    errors = errors + 1;
end

// Check that the SRAM uses the same mapping.
if (sram_bank0 != eb0 || sram_bank1 != eb1 ||
    sram_bank2 != eb2 ||
    sram_addr0 != ea0 || sram_addr1 != ea1 ||
    sram_addr2 != ea2) begin
    $display(
        "FAIL SRAM ADDRESS row=%0d col=%0d", r, c
    );
    errors = errors + 1;
end

end
endtask

// Compare the synchronous 24-bit SRAM output against
// three independently calculated reference pixels.
task check_pixels;
    input [7:0] r;
    input [7:0] c;
    reg [23:0] expected_word;
    begin
        expected_word = {
            exp_pixel(r,c),
            exp_pixel(r,c+8'd1),
            exp_pixel(r,c+8'd2)
        };

        pixel_checks = pixel_checks + 1;

        if (out_valid != 1'b1 ||
            cnn_data != expected_word) begin
            $display(
                "FAIL PIXELS row=%0d col=%0d expected=%h actual=%h valid=%b",
                r,c,expected_word,cnn_data,out_valid
            );
        end
    end
endtask

```

```

        );
        errors = errors + 1;
    end
    else begin
        $display(
            "PASS PIXELS row=%0d col=%0d data=%h",
            r,c,cnn_data
        );
    end
end
endtask

// Check signals sampled before the clock edge. SRAM registers
// data for those same row/column values at this edge.
always @(posedge clk) begin
    if (rst_n && read_en) begin
        sampled_row = row;
        sampled_col = col;

        check_address(sampled_row,sampled_col);

        #1;

        cycles = cycles + 1;

        if (sampled_row == 0 && sampled_col == 0)
            check_pixels(sampled_row,sampled_col);

        if (sampled_row == 7 && sampled_col == 12)
            check_pixels(sampled_row,sampled_col);

        // Verify progression to the next coordinate.
        if (sampled_col < 8'd253) begin
            if (row != sampled_row ||
                col != sampled_col + 8'd1) begin
                $display(
                    "FAIL SEQUENCE after row=%0d col=%0d",
                    sampled_row,sampled_col
                );
                errors = errors + 1;
            end
        end
        else if (sampled_row < 8'd255) begin
            if (row != sampled_row + 8'd1 || col != 8'd0) begin
                $display(
                    "FAIL ROW TRANSITION after row=%0d col=%0d",
                    sampled_row,sampled_col
                );
                errors = errors + 1;
            end
        end
    end
end

initial begin
    $dumpfile("part2_address_generator.vcd");
    $dumpvars(0,tb_part2_address_generator);

    $display("=====");
    $display("PART 2: ADDRESS GENERATOR VERIFICATION");
    $display("=====");

```

```

// Reset.
repeat (2) @(negedge clk);
rst_n = 1;

// Preload the six pixels required by the two checks.
load_pixel(8'd0,8'd0);
load_pixel(8'd0,8'd1);
load_pixel(8'd0,8'd2);

load_pixel(8'd7,8'd12);
load_pixel(8'd7,8'd13);
load_pixel(8'd7,8'd14);

// Start the address scan.
@(negedge clk);
start = 1;

@(negedge clk);
start = 0;

// Row 7, column 12 is reached after the scan advances.
wait (row == 8'd7 && col == 8'd12 && read_en);
// Allow the next rising edge to capture the target SRAM read.
@(posedge clk);
#2;

$display("-----");
$display("Address checks : %0d", address_checks);
$display("Pixel checks   : %0d", pixel_checks);
$display("Errors          : %0d", errors);

if (errors == 0 && pixel_checks == 2)
    $display("PART 2 TEST PASSED");
else
    $display("PART 2 TEST FAILED");

$display("-----");
$finish;
end

// Safety timeout if the scan never reaches the target.
initial begin
    #200000;
    $display("TIMEOUT: address scan did not finish in time");
    $finish;
end

endmodule

```

#### 4. terminal

```
sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/IITKGP_assignment/3$ iverilog -g2012 -Wall \  
-s tb_part2_address_generator \  
-o /tmp/part2.out \  
cnn_address_generator.sv \  
cnn_input_sram.sv \  
tb_part2_address_generator.sv \  
&& vvp /tmp/part2.out
```

VCD info: dumpfile part2\_address\_generator.vcd opened for output.

PART 2: ADDRESS GENERATOR VERIFICATION

PASS PIXELS row=0 col=0 data=11161b

PASS PIXELS row=7 col=12 data=62676c

Address checks : 1791

Pixel checks : 2

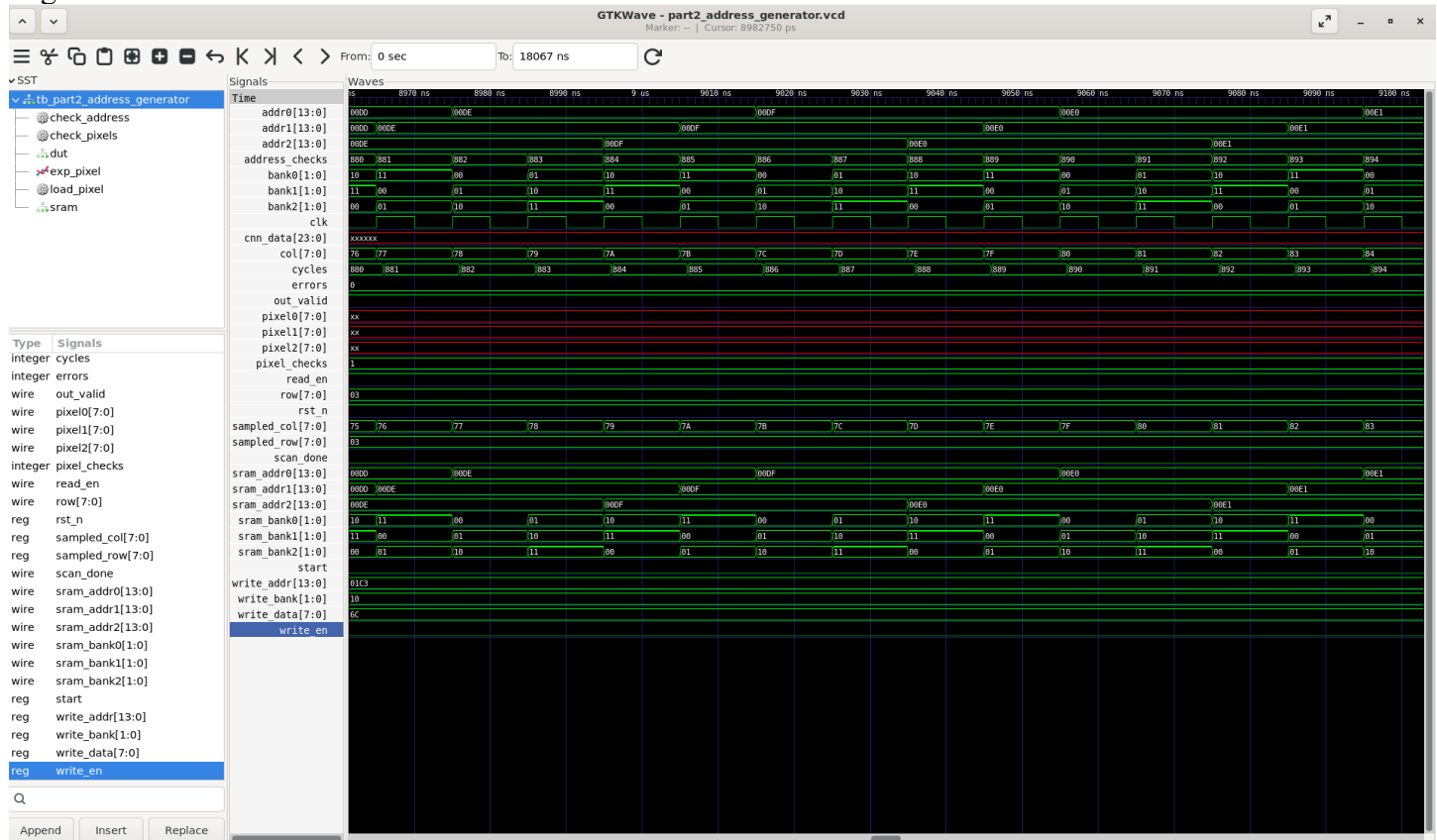
Errors : 0

PART 2 TEST PASSED

tb\_part2\_address\_generator.sv:278: \$finish called at 18067000 (1ps)

```
sureshvlsi@DESKTOP-CB90UTN:/mnt/c/Users/Lenovo/IITKGP_assignment/3$
```

#### 5. gtkwave



### Part3:

#### 1. cnn\_conv\_window\_fifo.sv

```
`timescale 1ns/1ps

module cnn_conv_window_fifo (
    input wire          clk,
    input wire          rst_n,
    input wire          pixel_valid,
    input wire signed [7:0] pixel_in,

    input wire signed [7:0] kernel0,
    input wire signed [7:0] kernel1,
    input wire signed [7:0] kernel2,
    input wire signed [7:0] kernel3,
    input wire signed [7:0] kernel4,
    input wire signed [7:0] kernel5,
    input wire signed [7:0] kernel6,
    input wire signed [7:0] kernel7,
    input wire signed [7:0] kernel8,

    output wire signed [7:0] p00,
    output wire signed [7:0] p01,
    output wire signed [7:0] p02,
    output wire signed [7:0] p10,
    output wire signed [7:0] p11,
    output wire signed [7:0] p12,
    output wire signed [7:0] p20,
    output wire signed [7:0] p21,
    output wire signed [7:0] p22,

    output wire          win_valid_next,
    output reg           out_valid,
    output reg signed [23:0] mac_sum
);

    reg signed [7:0] fifo [0:8];
    reg [3:0] fifo_count;
    integer i;

    // fifo[0] is the oldest sample; fifo[8] is the newest.
    assign p00 = fifo[0];
    assign p01 = fifo[1];
    assign p02 = fifo[2];
    assign p10 = fifo[3];
    assign p11 = fifo[4];
    assign p12 = fifo[5];
    assign p20 = fifo[6];
    assign p21 = fifo[7];
    assign p22 = fifo[8];

    // High when accepting the current input will produce a full 9-pixel window.
    assign win_valid_next = pixel_valid && (fifo_count >= 4'd8);

    // Calculate the window that will exist AFTER the current accepted pixel.
    // This lets the ninth input participate in the first complete MAC result.
    wire signed [7:0] q0 = (fifo_count >= 4'd8) ? fifo[1] : fifo[0];
    wire signed [7:0] q1 = (fifo_count >= 4'd8) ? fifo[2] : fifo[1];
    wire signed [7:0] q2 = (fifo_count >= 4'd8) ? fifo[3] : fifo[2];
    wire signed [7:0] q3 = (fifo_count >= 4'd8) ? fifo[4] : fifo[3];
    wire signed [7:0] q4 = (fifo_count >= 4'd8) ? fifo[5] : fifo[4];
    wire signed [7:0] q5 = (fifo_count >= 4'd8) ? fifo[6] : fifo[5];
```

```

wire signed [7:0] q6 = (fifo_count >= 4'd8) ? fifo[7] : fifo[6];
wire signed [7:0] q7 = (fifo_count >= 4'd8) ? fifo[8] : fifo[7];
wire signed [7:0] q8 = pixel_in;

wire signed [15:0] prod0 = q0 * kernel0;
wire signed [15:0] prod1 = q1 * kernel1;
wire signed [15:0] prod2 = q2 * kernel2;
wire signed [15:0] prod3 = q3 * kernel3;
wire signed [15:0] prod4 = q4 * kernel4;
wire signed [15:0] prod5 = q5 * kernel5;
wire signed [15:0] prod6 = q6 * kernel6;
wire signed [15:0] prod7 = q7 * kernel7;
wire signed [15:0] prod8 = q8 * kernel8;

wire signed [23:0] sum_next =
    {{8{prod0[15]}},prod0} + {{8{prod1[15]}},prod1} +
    {{8{prod2[15]}},prod2} + {{8{prod3[15]}},prod3} +
    {{8{prod4[15]}},prod4} + {{8{prod5[15]}},prod5} +
    {{8{prod6[15]}},prod6} + {{8{prod7[15]}},prod7} +
    {{8{prod8[15]}},prod8};

always @(posedge clk) begin
    if (!rst_n) begin
        fifo_count <= 0;
        out_valid <= 0;
        mac_sum <= 0;
        for (i = 0; i < 9; i = i + 1)
            fifo[i] <= 0;
    end else begin
        out_valid <= 0;
        if (pixel_valid) begin
            for (i = 0; i < 8; i = i + 1)
                fifo[i] <= fifo[i+1];
            fifo[8] <= pixel_in;

            if (fifo_count < 9)
                fifo_count <= fifo_count + 1'b1;

            if (fifo_count >= 8) begin
                mac_sum <= sum_next;
                out_valid <= 1;
            end
        end
    end
end
endmodule

```

## 2. tb\_part3\_conv\_window\_fifo.sv

```

`timescale 1ns/1ps

module tb_part3_conv_window_fifo;
    reg clk = 0;
    reg rst_n = 0;
    reg pixel_valid = 0;
    reg signed [7:0] pixel_in = 0;

    // Kernel: [1 0 -1; 1 0 -1; 1 0 -1]
    reg signed [7:0] k0 = 1, k1 = 0, k2 = -1;
    reg signed [7:0] k3 = 1, k4 = 0, k5 = -1;
    reg signed [7:0] k6 = 1, k7 = 0, k8 = -1;

```

```

wire signed [7:0] p00,p01,p02,p10,p11,p12,p20,p21,p22;
wire win_valid_next;
wire out_valid;
wire signed [23:0] mac_sum;

integer errors = 0;
integer output_checks = 0;
integer n;
reg expected_win_next;

cnn_conv_window_fifo dut (
    .clk(clk), .rst_n(rst_n),
    .pixel_valid(pixel_valid), .pixel_in(pixel_in),
    .kernel0(k0), .kernel1(k1), .kernel2(k2),
    .kernel3(k3), .kernel4(k4), .kernel5(k5),
    .kernel6(k6), .kernel7(k7), .kernel8(k8),
    .p00(p00), .p01(p01), .p02(p02),
    .p10(p10), .p11(p11), .p12(p12),
    .p20(p20), .p21(p21), .p22(p22),
    .win_valid_next(win_valid_next),
    .out_valid(out_valid), .mac_sum(mac_sum)
);

always #5 clk = ~clk;

task send_pixel;
    input integer value;
    input integer index;
    begin
        @(negedge clk);
        pixel_valid = 1;
        pixel_in = value;
        #1;
        expected_win_next = (index >= 8);
        if (win_valid_next != expected_win_next) begin
            $display("FAIL win_valid_next at input index %0d: got %b expected %b", index,
win_valid_next, expected_win_next);
            errors = errors + 1;
        end
        @(posedge clk);
        #1;
        if (index >= 8) begin
            output_checks = output_checks + 1;
            if (out_valid != 1'b1) begin
                $display("FAIL out_valid at input index %0d", index);
                errors = errors + 1;
            end
            // With this vertical-edge kernel, each 9-sample window
            // of consecutive integers 1..9, then 2..10, sums to -6.
            if (mac_sum != -24'sd6) begin
                $display("FAIL mac_sum at input index %0d: got %0d expected -6", index,
mac_sum);
                errors = errors + 1;
            end else begin
                $display("PASS window ending at input %0d: mac_sum=%0d", index+1, mac_sum);
            end
        end else if (out_valid != 1'b0) begin
            $display("FAIL out_valid asserted before full window at index %0d", index);
            errors = errors + 1;
        end
    end
end

```

```

endtask

initial begin
    $dumpfile("part3_conv_window_fifo.vcd");
    $dumpvars(0,tb_part3_conv_window_fifo);
    repeat (2) @(negedge clk);
    rst_n = 1;

    for (n = 1; n <= 10; n = n + 1)
        send_pixel(n, n-1);

    @(negedge clk);
    pixel_valid = 0;

    $display("-----");
    $display("Part 3 output checks : %0d", output_checks);
    $display("Errors                : %0d", errors);
    if (errors == 0 && output_checks == 2)
        $display("PART 3 STANDALONE WINDOW/MAC TEST PASSED");
    else
        $display("PART 3 STANDALONE WINDOW/MAC TEST FAILED");
    $display("-----");
    $finish;
end

initial begin
    #5000;
    $display("TIMEOUT");
    $finish;
end
endmodule

```

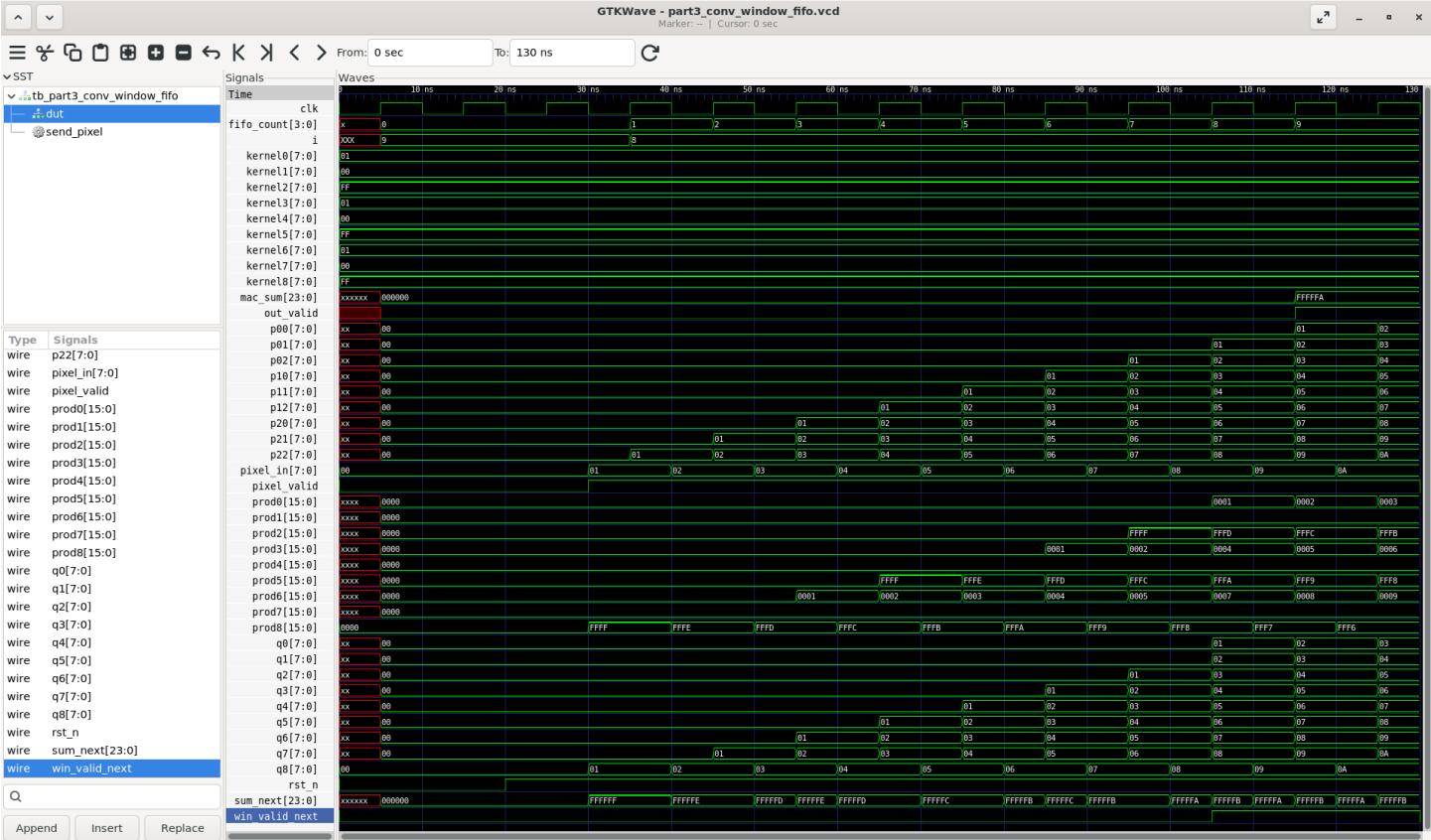
### 3. terminal

```

sureshvlasi@DESKTOP-CB90UTN: /mnt/c/Users/Lenovo/IITKGP_assignment/3$ iverilog -g2012 -wall
-s tb_part3_conv_window_fifo \
-o /tmp/part3.out \
cnn_conv_window_fifo.sv \
tb_part3_conv_window_fifo.sv \
&& vvp /tmp/part3.out
VCD info: dumpfile part3_conv_window_fifo.vcd opened for output.
PASS window ending at input 9: mac_sum=-6
PASS window ending at input 10: mac_sum=-6
-----
Part 3 output checks : 2
Errors                : 0
PART 3 STANDALONE WINDOW/MAC TEST PASSED
-----
tb_part3_conv_window_fifo.sv:95: $finish called at 130000 (1ps)
sureshvlasi@DESKTOP-CB90UTN: /mnt/c/Users/Lenovo/IITKGP_assignment/3$ 

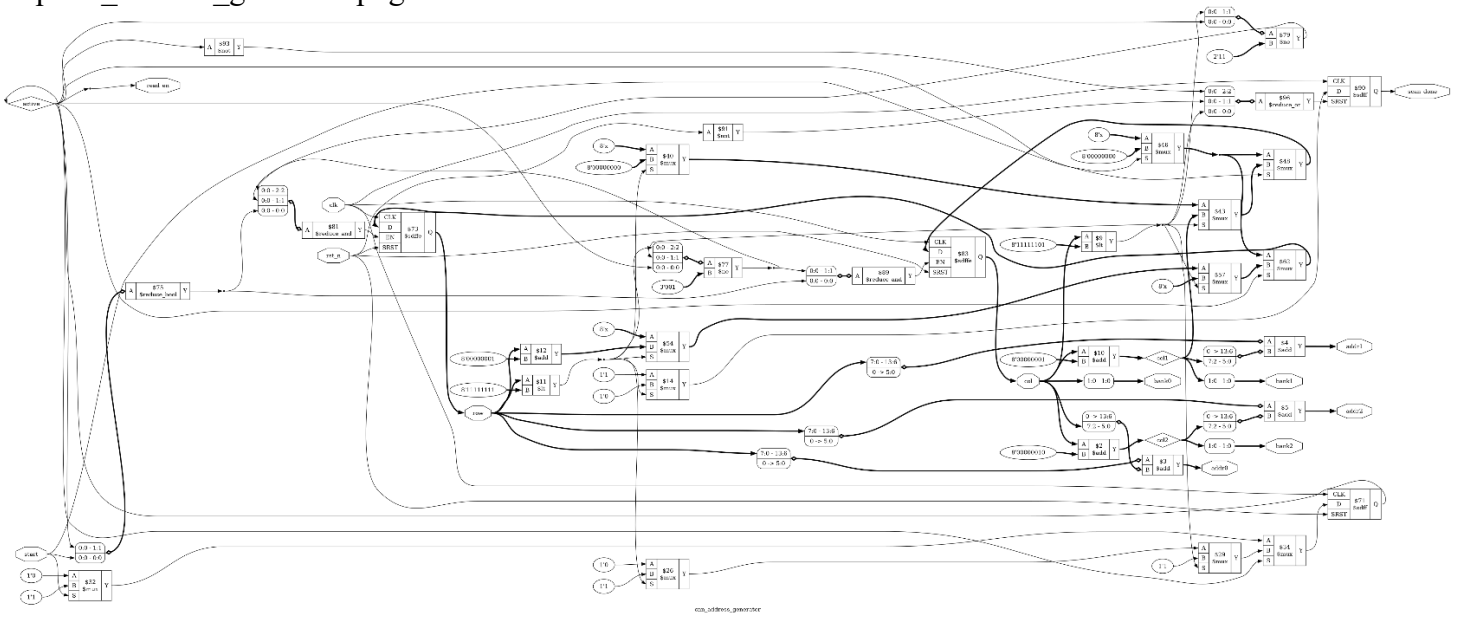
```

4. gtkwave



#### Part4:

1. cnn\_address\_generator.sv
2. cnn\_input\_sram.sv
3. part4\_address\_generator.dot
4. part4\_address\_generator.png



5. part4\_input\_sram.dot
6. part4\_input\_sram.png

